

{ }

• Java •



Java Streams

Complete Interview Guide



Advanced Stream API
Every Senior Java Developer
Should Know



Save Before
Your Interview

@java_interview_prep

Slide 2

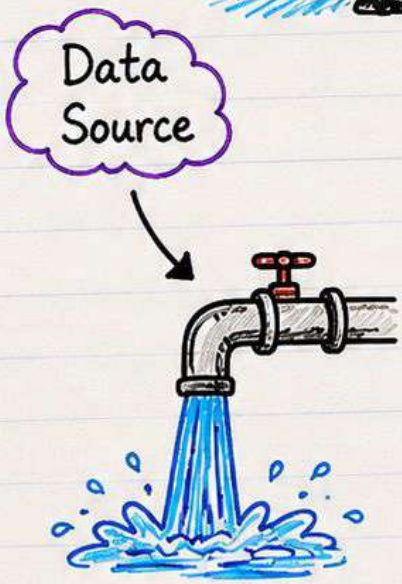
What is Stream?



Stream is NOT a Collection



- ☑ Doesn't store data
- ☑ Processes data from a source
- ☑ Supports functional programming
- ☑ Lazy evaluation
- ☑ Internal Iteration
- ☑ Can be consumed only once




```
list.stream()
    .filter(emp → emp.getSalary() > 50000)
    .map(Employee::getName)
    .toList();
```

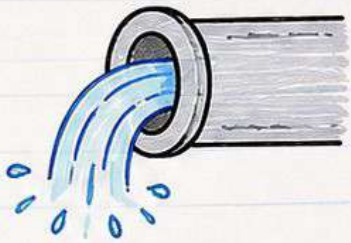


Interview Tip

A Stream is a pipeline of operations, not a data structure.



3 Stream Pipeline Explained



1 SOURCE

The data source from where stream originates.

- Collection (List, Set, Map)
- Array
- I/O channels, files
- Generator functions (Stream.iterate, generate)

2 INTERMEDIATE OPERATIONS

Transform the stream into another stream They are LAZY. Execution happens only when a terminal operation is invoked.

- filter()
- map() / flatMap()
- sorted()
- distinct()
- limit(n) / skip(n)
- peek()
- ... and many more

Lazy means nothing is processed until a terminal operation is called.

3 TERMINAL OPERATIONS

Terminate the pipeline and produce a result or side-effect.

- collect()
- forEach() / forEachOrdered()
- reduce()
- count()
- anyMatch(), allMatch(), noneMatch()
- findFirst(), findAny()
- ... and many more



Example

```
List<Employee> list = ...
list.stream()                                          // Source
        .filter(emp -> emp.getSalary() > 50000)        // Intermediate
        .map(Employee::getName)
        .sorted()
        .collect(Collectors.toList());                 // Terminal
```

Key Point

Intermediate operations return a Stream.
Terminal operation produces a result.



Save this post before your interview!

4 Lazy Evaluation (Most Asked)



Nothing executes here:

```
list.stream()
    .filter(this::isActive)
    .map(Employee::getName);
```

---> Just building the pipeline

Execution starts here:

```
.collect(Collectors.toList());
```

---> Terminal operation triggers the execution

Why?

- ☑ Better performance
- ☑ Optimized execution
- ☑ Avoid unnecessary processing
- ☑ Short-circuiting is possible

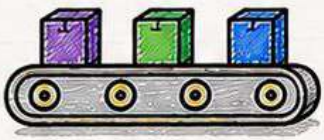


Interview Question

Why doesn't filter() execute immediately?

Answer:

Because intermediate operations are lazy and only build the execution pipeline.



Pro Tip Intermediate ops are lazy, terminal ops are eager!

Save this post before your interview!

@java_interview_prep

5 Parallel Stream

(Tricky Interview Question)



`list.parallelStream()` → Runs the stream operations in parallel using ForkJoinPool.commonPool()



Benefits

- ☑ Improves performance for CPU-intensive tasks
- ☑ Best for large datasets
- ☑ Breaks data into smaller chunks and processes in parallel
- ☑ Utilizes multiple CPU cores efficiently

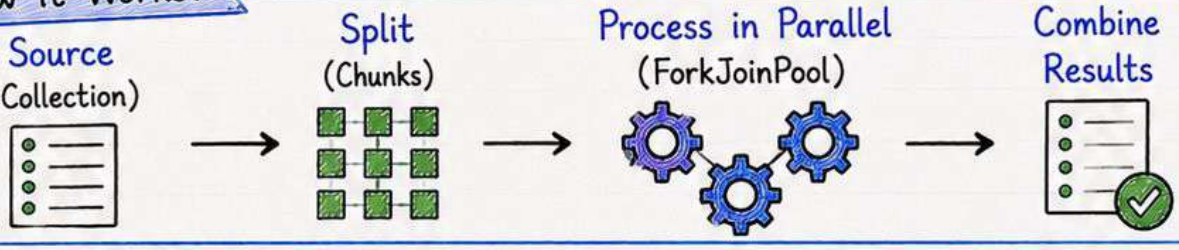


Problems / Pitfalls

- ✗ Not suitable for small datasets (overhead > benefit)
- ✗ Shared mutable state can cause incorrect results
- ✗ Order of execution is not guaranteed
- ✗ Blocking I/O operations reduce performance
- ✗ Debugging is harder



How it Works?



When to Use?

- ★ Large datasets (thousands of records+)
- ★ CPU-bound computations
- ★ Independent operations (no shared mutable state)



Interview Answer Tip

☆ Never use Parallel Stream blindly. Always measure performance and ensure thread-safety.

Pro Tip If you need ordering, use `stream()`. If order doesn't matter, `parallelStream()` can give better performance.



6 Collectors

Every Senior Developer Must Know



Most Important Collectors

Used with `collect()` to transform Stream output

1 `groupingBy()`	Groups elements by a classifier function.
2 `partitioningBy()`	Divides data into two groups (true / false).
3 `mapping()`	Applies a mapping function before collecting.
4 `joining()`	Concatenates elements into a String.
5 `counting()`	Counts the number of elements.
6 `summarizingInt()` / `summarizingLong()` / `summarizingDouble()`	Provides summary statistics like sum, avg, min, max, count.
7 `toMap()`	Converts elements into a Map.
8 `collectingAndThen()`	Applies a finishing transformation to the result.
9 `toList()` / `toSet()` / `toCollection()`	Collects elements into a List, Set or any Collection.

Example

```
Map<String, List<Employee>> empByDept =
    employees.stream()
        .collect(Collectors.groupingBy(Employee::getDepartment));
```

Group employees by department

Real-World Use Cases

- ☆ Group orders by customer
- ☆ Transactions by date
- ☆ Products by category
- ☆ Students by grade
- ☆ Employees by department
- ☆ Logs by log level

Interview Tip

Know when and why to use each collector.

7 Reduce vs Collect



Two ways to produce a result from a Stream

REDUCE

Produces a single result

Example

```
List<Integer> numbers = List.of(1, 2, 3, 4, 5);
int sum = numbers.stream()
        .reduce(0, Integer::sum);
// sum = 15
```

Key Points

- Returns one value (immutable result)
- Best for aggregation
- Can use identity value
- Overloads available
  1. reduce(identity, accumulator)
  2. reduce(accumulator)
  3. reduce(identity, accumulator, combiner)



COLLECT

Produces a mutable result

Example

```
List<String> names = List.of("A", "B", "C");
List<String> list = names.stream()
            .collect(Collectors.toList());
// list = [A, B, C]
```

Key Points

- Returns a collection or object
- Best for transforming data into collections / maps / etc.
- Uses Collector
- More flexible (many built-in collectors)



Quick Comparison

Aspect	Reduce	Collect
Returns	Single value	Collection / Object
Mutability	Immutable result	Mutable container
Use case	Aggregation, computation	Collection, grouping, transformation
Example	Sum, Max, Min, Count	List, Set, Map, Grouping
Method	reduce()	collect(Collector)

When to Use?

Use reduce() when you need one final answer.

Use collect() when you need a collection or complex result.

Pro Tip

If you want ONE final value → use reduce().
If you want MANY values or a structure → use collect().



@java_interview_prep

8 Senior-Level Interview Questions



1 What is Stream in Java? →	A Stream is a sequence of elements from a source that supports aggregate operations. It is not a data structure and does not store data.
2 Why are Streams single-use? →	A Stream has a pipeline and a source which gets consumed after a terminal operation. Reusing it would mean re-consuming the source.
3 What is the difference between map() and flatMap()? →	• map() transforms each element into another object. • flatMap() first maps each element to a Stream and then flattens them into a single Stream.
4 What is the difference between findFirst() and findAny()? →	• findFirst() returns the first element according to encounter order. • findAny() returns any element; does not guarantee which one. Better for parallel streams.
5 How does Parallel Stream work internally? →	It uses ForkJoinPool.commonPool() to split the data into multiple chunks, process them in parallel and then combine the results.
6 When should Parallel Stream be avoided? →	For small datasets, I/O bound tasks, stateful operations, ordered streams and when using shared mutable state.
7 Why is Stream lazy? →	Intermediate operations are lazy to enable optimizations, short-circuiting and to avoid unnecessary work.
8 reduce() vs collect()? →	• reduce() produces a single result (e.g., sum, max). • collect() accumulates results into a collection/structure.
9 How does groupingBy() work? →	It is a Collector that groups elements based on a classifier function and returns a Map<K, List<T>>.
10 Why shouldn't Streams modify shared state? →	Because it can lead to race conditions, inconsistent results and is not thread-safe, especially in parallel streams.

Pro Tip

Prefer immutable operations, avoid side effects and always measure performance before using parallel streams.





Save this post before your interview!

@java_interview_prep